

Informe de Implementación: Sistema de Gestión de Tareas Georreferenciadas

Asignatura: Taller de Base de Datos 1-2026, Control 2

Integrantes:

- Bruno Gutiérrez
- Alejandra Silva
- Emilio Poblete
- Bastian Ramos
- Jesús Ganga

Introducción:

Este informe muestra cómo se creó un sistema para gestionar tareas que puede analizar lugares en un mapa. El objetivo es que los usuarios puedan organizar sus tareas de manera efectiva y obtener información sobre sus datos en relación con lugares específicos.

El sistema se diseñó de manera que la parte que los usuarios ven y la parte que hace el trabajo estén separadas. Esto se logró mediante una API RESTful, el sistema cumple con requisitos básicos como proteger la información de los usuarios, registrar tareas y hacer análisis avanzados de lugares (con PostGIS).

Arquitectura del Sistema:

El sistema se ha diseñado con una arquitectura en varias capas y se ha implementado utilizando contenedores con Docker Compose este se divide en tres partes principales:

- **Nivel de Frontend:** Esta parte se ha creado con Vue.js 3 y utiliza componentes reutilizables para mantener una apariencia coherente. Se utilizan herramientas como Leaflet.js y OpenStreetMap para mostrar puntos geográficos.
- **Nivel de Backend:** Este nivel se ha desarrollado con Spring Boot y actúa como el corazón del sistema mediante una API RESTful.
- **Nivel de Base de datos:** Aquí se utiliza una base de datos llamada PostgreSQL, que cuenta con la extensión PostGIS. Esto permite tratar las ubicaciones geográficas como objetos geométricos, lo que facilita la realización de consultas complejas sobre proximidad.

Modelo de Datos Geoespacial:

El modelo de datos está formado por tres entidades principales que se relacionan entre sí, Esto ayuda a hacer más eficientes las consultas que involucran ubicaciones geográficas.

- **Usuario (User):** Aquí se guardan los datos de acceso. Cada tarea y punto geográfico está asociado a un usuario. Esto asegura que la información de cada usuario esté ah aislada y sea segura.

- Punto Geográfico (GeoPoint): Esta entidad es fundamental para el análisis de ubicaciones. Utiliza un sistema de referencia espacial. Esto permite realizar análisis precisos.
- Tarea (Task): Representa una unidad de trabajo. Está directamente relacionada con un punto geográfico. Esta relación permite calcular distancias y concentraciones de manera directa en la base de datos. Esto hace que el sistema sea más rápido y eficiente.

Seguridad Implementada:

Para proteger el sistema y los datos de los usuarios, utilizamos tokens JWT (JSON Web Tokens) para autenticar y autorizar el acceso. Cada vez que un usuario hace una petición, verificamos su identificador de usuario (userId) de manera estricta. Esto nos permite asegurarnos de que un usuario solo pueda ver, cambiar o eliminar las tareas que son suyas. Además, usamos Spring Security para prevenir ataques comunes como CSRF. Y con los repositorios JPA, evitamos vulnerabilidades de inyección SQL.

Preguntas a responder:

A continuación, se detallan las consultas espaciales nativas utilizadas para dar respuesta a los requerimientos del negocio.

- ¿Cuántas tareas ha hecho el usuario por sector?

R: Se realiza un conteo agrupando por el atributo de sector asociado a las tareas completadas del usuario.

```
SELECT gp.sector AS sector, COUNT(ut.id_task) AS total_tareas
FROM user_tasks ut
JOIN task t ON ut.id_task = t.id_task
JOIN geo_point gp ON t.id_geo_point = gp.id_geo_point
WHERE ut.id_user = :idUser
GROUP BY gp.sector
ORDER BY total_tareas DESC;
```

- ¿Cuál es la tarea más cercana al usuario (que esté pendiente)?

R: Se utiliza la función ST_Distance para calcular la distancia entre el punto de origen del usuario y las coordenadas de las tareas con estado "Pendiente", ordenando los resultados de menor a mayor.

```
SELECT t.* FROM task t
JOIN geo_point gp_task ON t.id_geo_point = gp_task.id_geo_point
CROSS JOIN users u
JOIN geo_point gp_user ON u.id_geo_point = gp_user.id_geo_point
WHERE u.id_user = :idUser
AND t.status = 'Pendiente'
ORDER BY ST_Distance(gp_task.point::geography, gp_user.point::geography) ASC
LIMIT 1;
```

- ¿Cuál es el sector con más tareas completadas en un radio de 2 kilómetros del usuario?

R: Se utiliza la función espacial ST_DWithin para filtrar únicamente las geometrías que se encuentran dentro de un radio de 2000 metros desde la ubicación de registro del usuario, agrupando posteriormente por sector.

```
SELECT gp_task.sector AS sector, COUNT(t.id_task) AS total_tareas
FROM task t
JOIN geo_point gp_task ON t.id_geo_point = gp_task.id_geo_point
CROSS JOIN users u
JOIN geo_point gp_user ON u.id_geo_point = gp_user.id_geo_point
WHERE u.id_user = :idUser
AND t.status = 'Completada'
AND ST_DWithin(gp_task.point::geography, gp_user.point::geography,
:radiusInMeters)
GROUP BY gp_task.sector
ORDER BY total_tareas DESC
LIMIT 1;
```

- ¿Cuál es el promedio de distancia de las tareas completadas respecto a la ubicación del usuario?

R: Se aplica la función de agregación AVG sobre el resultado de la función ST_Distance entre las coordenadas del usuario y sus tareas marcadas como completadas. Se tuvo precaución de filtrar estrictamente por el id_user para evitar promediar datos globales.

```
SELECT COALESCE(AVG(ST_Distance(gp_task.point::geography,
gp_user.point::geography)), 0)
FROM user_tasks ut
JOIN task t ON ut.id_task = t.id_task
JOIN geo_point gp_task ON t.id_geo_point = gp_task.id_geo_point
CROSS JOIN users u
JOIN geo_point gp_user ON u.id_geo_point = gp_user.id_geo_point
WHERE u.id_user = :idUser
AND ut.id_user = :idUser;
```

- ¿En qué sectores geográficos se concentran la mayoría de las tareas pendientes? (utilizando agrupación espacial).

R: Se extraen los sectores con la mayor acumulación de tareas en estado "Pendiente" evaluando su agrupación espacial.

```
SELECT gp.sector AS sector, COUNT(t.id_task) AS total_pendientes
FROM task t
JOIN geo_point gp ON t.id_geo_point = gp.id_geo_point
WHERE t.status = 'Pendiente'
GROUP BY gp.sector
ORDER BY total_pendientes DESC;
```

- ¿Cuántas tareas ha realizado cada usuario por sector?

R:A diferencia de la consulta individual, aquí se agrupan los datos por el nombre de cada usuario registrado en el sistema y por los sectores de sus tareas correspondientes, realizando un conteo general que permite comparar la actividad global.

```
SELECT u.user_name AS usuario, gp.sector AS sector, COUNT(ut.id_task) AS total_tareas
FROM users u
JOIN user_tasks ut ON u.id_user = ut.id_user
JOIN task t ON ut.id_task = t.id_task
JOIN geo_point gp ON t.id_geo_point = gp.id_geo_point
GROUP BY u.user_name, gp.sector
ORDER BY usuario ASC, total_tareas DESC;
```

- ¿Cuál es el sector con más tareas completadas dentro de un radio de 5 km desde la ubicación del usuario?

R:Se utiliza la función ST_DWithin convirtiendo las geometrías a tipo "geography" para calcular la distancia métrica real sobre la curvatura de la tierra. Se establece un radio estático de 5000 metros, filtrando por tareas completadas y limitando el resultado al sector con mayor concentración.

```
SELECT gp_task.sector AS sector, COUNT(t.id_task) AS total_tareas
FROM user_tasks ut
JOIN task t ON ut.id_task = t.id_task
JOIN geo_point gp_task ON t.id_geo_point = gp_task.id_geo_point
JOIN users u ON ut.id_user = u.id_user
JOIN geo_point gp_user ON u.id_geo_point = gp_user.id_geo_point
WHERE u.id_user = :idUser
AND t.status = 'Completada'
AND ST_DWithin(gp_task.point::geography, gp_user.point::geography, 5000)
GROUP BY gp_task.sector
ORDER BY total_tareas DESC
LIMIT 1;
```

- ¿Cuál es el promedio de distancia entre las tareas completadas y el punto registrado del usuario?

R:Se calcula anidando la función espacial ST_Distance dentro de la función de agregación AVG. Se aplica un filtro estricto sobre el estado de la tarea (t.status = 'Completada') y se utiliza COALESCE para devolver 0 en caso de que el usuario aún no tenga tareas completadas, evitando retornos nulos en la API.

```
SELECT COALESCE(AVG(ST_Distance(gp_task.point::geography,
gp_user.point::geography)), 0) AS distancia_promedio_metros
FROM user_tasks ut
JOIN task t ON ut.id_task = t.id_task
JOIN geo_point gp_task ON t.id_geo_point = gp_task.id_geo_point
JOIN users u ON ut.id_user = u.id_user
JOIN geo_point gp_user ON u.id_geo_point = gp_user.id_geo_point
```

```
WHERE u.id_user = :idUser  
AND t.status = 'Completada';
```

Conclusiones:

Con el desarrollo de este proyecto, logramos integrar con éxito una interfaz en Vue.js con un backend en Spring Boot. Esto nos permitió cumplir con todos los requisitos del Control 2, la clave de nuestro trabajo fue implementar PostgreSQL con la extensión PostGIS. Al usar funciones espaciales como ST_Distance o ST_DWithin, pudimos resolver las preguntas del negocio directamente en la base de datos. Esto hizo que el sistema fuera mucho más eficiente.

Al final, utilizar Docker fue una excelente decisión para nuestro equipo. Nos evitó los problemas de compatibilidad que normalmente surgen en nuestros computadores cuando programamos. Gracias a esto, logramos entregar una aplicación que funciona correctamente.

README:

Integrantes

- Bruno Gutiérrez
- Alejandra Silva
- Emilio Poblete
- Bastian Ramos
- Jesús Ganga

Sistema de Gestión de Tareas Georreferenciadas - Control 2

Este proyecto corresponde a la implementación de un sistema de gestión de tareas con funcionalidades espaciales, desarrollado con una arquitectura separada:

- **Frontend** en Vue.js
- **Backend** como API RESTful en Spring Boot
- **Base de Datos** en PostgreSQL con la extensión PostGIS para el almacenamiento y análisis de datos geoespaciales.

Requisitos Previos

Gracias a la contenedorización del proyecto, los requisitos para levantar el sistema en cualquier entorno local se han simplificado enormemente. Es estrictamente necesario contar con:

- **Docker** y **Docker Compose** instalados en el sistema.
- **Git** (para clonar el repositorio).

(Nota para desarrollo local sin contenedores: Java 21, Maven, Node.js/npm y PostgreSQL 16 con PostGIS).

Instrucciones de Despliegue (Ejecución Automática)

El proyecto está configurado para levantar la Base de Datos, el Backend y el Frontend de manera orquestada con un solo comando, creando las tablas y configuraciones de forma automática.

Paso 1: Obtener el proyecto

Abra una terminal y clone el repositorio (o haga un pull si ya lo tiene):

```
git clone https://github.com/Bix0/TBD.git
```

Paso 2: Posicionarse en la carpeta raíz del proyecto

```
cd "TBD/controles/CONTROL 2"
```

Paso 3: Levantar los servicios

Ejecute el siguiente comando para construir las imágenes y levantar todos los contenedores en segundo plano:

```
docker-compose up -d --build
```

Paso 4: Acceder a la aplicación

Una vez que el comando termine y los contenedores estén en ejecución (puede verificarlo con `docker ps`), los servicios estarán disponibles en las siguientes rutas:

- **Frontend (Interfaz de Usuario):** Ingrese desde su navegador a `http://localhost:80` (o simplemente `http://localhost`).
- **Backend (API RESTful):** Corriendo internamente y expuesto en `http://localhost:8080`.
- **Base de Datos (PostgreSQL/PostGIS):** Corriendo en el puerto 5432 (Credenciales por defecto en `docker-compose.yml`: Usuario `adminControl`, Password `Elpro123#`, BD `backend-control2`).

Credenciales y Acceso Inicial

Debido a que el sistema se levanta con una base de datos limpia, para comenzar a probar la aplicación siga estos pasos:

1. Ingrese a `http://localhost:80`
2. Diríjase a la vista de ****Registro**** en el frontend.
3. Cree un usuario nuevo proporcionando un nombre, contraseña y marcando su ubicación en el mapa.
4. Inicie sesión con las credenciales recién creadas para acceder al panel de tareas.

Paso 5: Detener la aplicación

Para apagar el sistema y liberar los puertos sin perder la información estructurada, ejecute:

```
docker-compose down
```